

Lab 2: Primary-Backup Service — Design Document

Preface

Goals

- Provide a fault-tolerant key/value storage service that resilient to failure of primary or backup individually while the other role can be refilled.
- Support linearizable (exactly-once) semantics for all client operations (Get, Put, Append).
- Automatically recover from server failures by promoting the backup to primary and onboarding new backups from idle servers.

Desired Fault Model

- Tolerate the permanent crash of the primary server: the backup is promoted to primary and continues serving clients.
- Tolerate the permanent crash of the backup server: the primary continues operating and a new backup is recruited from idle servers.
- Tolerate temporary network partitions: a server that is isolated but not crashed may rejoin as a backup if the primary is still alive.
- **Not tolerated:** simultaneous crash of both primary and backup; crash of the ViewServer.

Challenges

- **Split-brain prevention:** After a view change, the old primary must not continue serving client requests as if it were still primary, as this could lead to divergent state between clients.
- **State synchronization:** When a new backup is recruited, it must receive a full copy of the primary's state before any subsequent operations are applied, without blocking client progress indefinitely.
- **Exactly-once semantics:** Client requests forwarded from primary to backup must not be applied more than once, even under retries.
- **View acknowledgment:** The ViewServer must not advance views until the current primary has acknowledged the current view, to prevent the ViewServer from racing ahead of the servers.

Assumptions

- The ViewServer never crashes.
- Server crashes are permanent (no restart).
- The network may delay, reorder, or drop messages, but does not corrupt them.
- At most one server is the active primary at any given time.
- Clients retry indefinitely until they receive a successful response.

Protocol

Kinds of Nodes

ViewServer

- A single, unreplicated node that tracks liveness of servers and maintains the authoritative sequence of views.

PBServer (key/value server)

- Temporary roles:
 - **Primary:** Handles all client requests directly; forwards every operation to the backup before acknowledging to the client; sends state transfers to newly-assigned backups.
 - **Backup:** Rejects direct client requests; accepts forwarded operations from the primary; accepts state transfer from the primary.
 - **Idle:** Neither primary nor backup; sends Pings to volunteer for backup assignment; rejects all client and forwarded requests.

PBClient

- Sends operations to the current primary; retries on error or timeout; queries ViewServer to refresh its cached view when needed.

State

ViewServer State

Field	Type	Initial Value	Meaning
<code>currentView</code>	View (viewnum, primary, backup)	<code>{STARTUP_VIEWNUM, null, null}</code>	The current view being served
<code>currentViewAked</code>	boolean	false	Whether the primary of <code>currentView</code> has sent a Ping with <code>currentView.viewnum</code>
<code>recentPingers</code>	set of server addresses	empty	Servers that pinged since the most recent <code>PingCheckTimer</code>
<code>previousPingers</code>	set of server addresses	empty	Servers that pinged between the two most recent <code>PingCheckTimer</code> s
<code>idleServers</code>	set of server addresses	empty	Servers that are pinging but are neither primary nor backup

Note: "alive" means the server appeared in `previousPingers` (i.e., pinged within the last `PingCheckTimer` interval). `recentPingers` is swapped into `previousPingers` on each `PingCheckTimer` fire.

PBServer State

Field	Type	Initial Value	Meaning
<code>currentView</code>	View	<code>{STARTUP_VIEWNUM, null, null}</code>	The latest view this server knows about
<code>role</code>	enum {PRIMARY, BACKUP, IDLE}	IDLE	This server's current role
<code>app</code>	AMOApplication	empty	The wrapped key/value application (provides exactly-once semantics)
<code>backupInitialized</code>	boolean	false	Whether the current backup has received a full state transfer
<code>pendingForward</code>	optional (AMOCmd, viewnum)	absent	An in-flight forward to the backup, awaiting acknowledgment

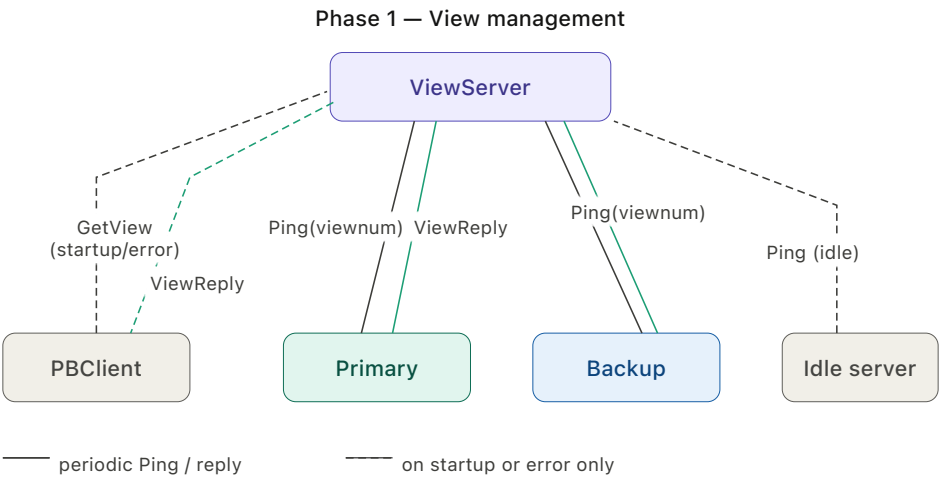
When `pendingForward` is present, the primary does not process new client requests until the forward is acknowledged. This ensures the backup applies operations in the same order as the primary.

PBClient State

Field	Type	Initial Value	Meaning
<code>cachedView</code>	View	<code>{STARTUP_VIEWNUM, null, null}</code>	Most recently known view, used to route requests
<code>pendingCommand</code>	optional AMOCmd	absent	The in-flight command awaiting a reply

Messages

Overview



Ping (PBServer → ViewServer)

- **Contents:** `{sender, viewnum}` — the sender's address and the latest view number it knows about.
- **When sent:** Spontaneously, once per `PING_MILLIS` (triggered by `PingTimer`). On startup, sent with `STARTUP_VIEWNUM`.
- **At ViewServer on receipt:**
 1. Add `sender` to `recentPingers`.
 2. Update `idleServers` to reflect current primary/backup assignments.
 3. If `sender == currentView.primary` && `viewnum == currentView.viewnum`, mark `currentViewAked = TRUE`.
 4. Reply with `ViewReply{currentView}`.
- **Notes:** The server is responsible for updating its own `currentView` and `role` from the `ViewReply`.

GetView (PBClient → ViewServer)

- **Contents:** `{sender}`
- **When sent:** On client startup, or when the client's `ClientTimer` fires (primary appears unresponsive), or when the client receives an error response.
- **At ViewServer on receipt:** Reply with `ViewReply{currentView}` without modifying any state.

ViewReply (ViewServer → PBServer / PBClient)

- **Contents:** `{view}` — the current view `{viewnum, primary, backup}`.
- **At PBServer on receipt:**
 1. If `view.viewnum > currentView.viewnum`, update `currentView` and `role` accordingly.
 2. If this server just became primary (was backup in prior view), set `backupInitialized = FALSE` and initiate a state transfer to the new backup if it is assigned.
 3. If this server just became backup, reset local `app` state in anticipation of receiving a state transfer.
- **At PBClient on receipt:** Update `cachedView`.

ClientRequest (PBClient → PBServer[primary])

- **Contents:** `{amoCommand}` where `amoCommand = {clientId, sequenceNum, operation}`.
- **When sent:** Spontaneously by client; retried periodically via `ClientTimer` until a successful reply.
- **At PBServer(primary) on receipt:**
 1. If `backupInitialized == FALSE` and a backup exists, return a temporary error (wait for state transfer to complete).
 2. Execute `amoCommand` against local `app` to get `result`.
 3. If a backup exists, set `pendingForward = (amoCommand, currentView.viewnum)` and send `ForwardRequest` to backup.

4. If no backup (or after ForwardAck received), reply to client with `ClientReply{result}`.

ForwardRequest (PBServer[primary] → PBServer[backup])

- **Contents:** `{amoCommand, viewnum}` - the operation to replicate and the view in which it is being forwarded.
- **When sent:** After the primary executes an operation locally and before replying to the client.
- **At PBServer(backup) on receipt:**
 1. If `viewnum != currentView.viewnum`, reply with error `VIEW_MISMATCH`.
 2. Otherwise, apply `amoCommand` to local `app` (AMOApplication deduplicates if already applied) and reply with `ForwardAck{ok}`.
- **At PBServer(not backup) on receipt:** Reply with error indicating this server is not the current backup.
- **At primary on receipt of ForwardAck{ok}:**
 1. Clear `pendingForward`.
 2. Send `ClientReply` to the waiting client.
- **At primary on receipt of ForwardAck{error: VIEW_MISMATCH}:** The primary infers a view change has occurred. It should stop treating itself as primary and return an error to the client so the client will query the ViewServer.

StateTransfer (PBServer[primary] → PBServer[backup])

- **Contents:** `{appSnapshot, viewnum}` — a full serialized copy of the primary's `AMOApplication` state and the view in which the transfer is occurring.
- **When sent:** When the primary detects a new backup has been assigned (`backupInitialized == FALSE` and `currentView.backup != null`). Retried via `StateTransferTimer` until a valid acknowledgment is received.
- **At PBServer(backup) on receipt:**
 1. If `viewnum < currentView.viewnum`, ignore as stale.
 2. If `viewnum > currentView.viewnum`, ignore (or defer) until local view catches up.
 3. If `viewnum == currentView.viewnum`, replace local `app` with `appSnapshot`.
 4. Reply with `StateTransferAck{viewnum}`.

StateTransferAck (PBServer[backup] → PBServer[primary])

- **Contents:** `{viewnum}`.
- **At primary on receipt:**
 1. Accept only if:
 - sender is `currentView.backup`,
 - `viewnum == currentView.viewnum`,
 - and `backupInitialized == FALSE` (i.e., transfer is still pending).

2. If accepted, set `backupInitialized = TRUE`.
 3. Cancel `StateTransferTimer`.
 4. Begin/continue forwarding client requests to backup.
-

Timers

PingTimer (PBServer)

- **Pattern:** Tick pattern.
- **Contents:** None.
- **Set:** On server initialization; reset every time it fires.
- **On fire:** Send `Ping{self, currentView.viewnum}` to ViewServer. If this server is the primary of a view that has not yet be acknowledged, send `Ping` with the previous view number until ready.

PingCheckTimer (ViewServer)

- **Pattern:** Tick pattern.
- **Contents:** None.
- **Set:** On ViewServer initialization; reset every time it fires.
- **On fire:**
 1. Move `recentPingers` to `previousPingers`; clear `recentPingers`.
 2. Determine liveness: a server is alive if it is in `previousPingers`.
 3. If `currentViewAked`:
 - If primary is dead: promote backup to primary; pick an idle server in `idleServers` as new backup if available; increment viewnum; set `currentViewAked = FALSE`.
 - Else if backup is dead or absent and an idle server is available: assign idle server as backup; increment viewnum; set `currentViewAked = FALSE`.
 4. If `!currentViewAked`, do not change views regardless of liveness.

ClientTimer (PBClient)

- **Pattern:** Tick pattern.
- **Contents:** None.
- **Set:** On client initialization; reset on fire.
- **On fire:** If `pendingCommand` is present and no reply received, send `GetView` to ViewServer to refresh `cachedView`, then retry same `amoCommand` to the new primary.

StateTransferTimer (PBServer[primary])

- **Pattern:** Discard pattern.
- **Contents:** `{targetBackup, viewnum}` - the backup address and viewnum at the time the transfer was initiated.
- **Set:** `backupInitialized == FALSE` and `currentView.backup != null`.
- **On fire:**
 1. If `currentView.viewnum != viewnum` OR `currentView.backup != targetBackup`, ignore.
 2. If `backupInitialized == TRUE`, ignore (already done).
 3. Otherwise, resend `StateTransfer` to `targetBackup`.

Correctness / Liveness Analysis

Failure Table

Message	Delayed	Dropped	Reordered	Duplicated
Ping	ViewServer may miss a liveness window but will catch it on the next PingCheckTimer	Server is considered dead after 2 missed windows; client retries eventually find the new primary	Out-of-order Pings carry their viewnum; ViewServer uses the viewnum to determine ack status	Extra Pings update liveness bookkeeping harmlessly
ViewReply	Server/client uses its cached view until reply arrives; no stale operation is served	Server retries on next PingTimer; client retries on ClientTimer	Old ViewReplies have lower viewnums and are ignored	No effect if viewnum matches; ignored if lower
ClientRequest	Client retries via ClientTimer	Client retries via ClientTimer	AMO deduplication prevents double-execution	AMO seqnum deduplication handles exact duplicate
ForwardRequest	Primary holds reply to client until ForwardAck; no data loss	Primary retries via its pending forward mechanism; client waits	AMO deduplication at backup prevents double-apply	AMO handles duplicate forwards
StateTransfer	Backup waits; primary resends via StateTransferTimer	StateTransferTimer resends; backupInitialized stays false	Old transfers carry viewnum; stale ones ignored	Applying twice leaves backup in same state

Node Crash Analysis

Primary crashes:

The ViewServer detects the primary's absence after two consecutive `PingCheckTimer` s. Since `currentViewAked` must be true before a view change, the backup is promoted to primary. Clients that still have the old primary cached will receive no response and will query the ViewServer on their `ClientTimer`, thus finding the new primary.

Backup crashes:

The primary continues serving clients alone. `backupInitialized` is reset to FALSE if the ViewServer assigns a new backup. The primary suspends client processing until the state transfer to the new backup completes, then resumes.

Idle server crashes:

No impact on correctness. The ViewServer uses `idleServers` as the pool of candidates for backup promotion.

ViewServer:

In lab2 we assume it never crash. This is an acknowledged limitation.

Partition / Reconnect Analysis

Old primary partitioned (not crashed):

The ViewServer promotes the backup (S2) to primary in view `i+1`. S1 still believes it is primary for view `i`. When S1 forwards a client request to S2, S2 is now operating in view `i+1` as primary, not backup, so it rejects the forwarded request with `VIEW_MISMATCH`. S1 then returns an error to the client, which queries the ViewServer and discovers S2 is the new primary. This prevents split-brain.

Backup partitioned (not crashed):

The primary detects that the ForwardRequest to the backup is not acknowledged. It may retry a bounded number of times, but if the backup remains unreachable, the primary may operate without a backup. When the partitioned server reconnects, it will ping the ViewServer with a stale viewnum; the ViewServer will assign it as idle or backup depending on availability.

Both primary and backup partitioned from each other but reachable by ViewServer:

The ViewServer will eventually declare the backup dead. It will then advance the view to make the primary operate alone. This requires the primary to have previously acked the current view.

Conclusion

Goals Achieved

- **Fault tolerance for single-server failure:** The ViewServer detects failure via missed Pings and advances the view. The protocol rules that when there is a backup and the primary fails, promotion is to that backup, and the primary's acknowledgement rule prevents the ViewServer from racing ahead.
- **Linearizability / exactly-once:** All operations are deduplicated via `AMOApplication` on both primary and backup. The primary does not reply to the client until the backup has applied the operation, ensuring both replicas remain consistent.
- **No split-brain:** The `VIEW_MISMATCH` rejection mechanism and the ViewServer's acknowledgement rule together ensure at most one server acts as primary at any time.

Limitations

- **ViewServer is a single point of failure.** Once the ViewServer crashes, no view changes can occur and the system becomes unavailable to new failures.
- **Full state transfer on backup change.** A new backup must receive the complete application snapshot, which can be slow for large datasets; incremental catch-up is not supported.
- **Operations are serialized.** The primary processes one operation at a time (waiting for ForwardAck before proceeding), limiting throughput.

- **No disk persistence.** Simultaneous crash of both primary and backup results in permanent data loss.
- **Stuck view.** If the primary crashes before acknowledging the view in which it is primary, the ViewServer cannot advance and the system is permanently unavailable.